

Isaac Gym vs. Isaac Lab: what actually changes?

Both support GPU-parallel robot simulation. The main difference here is how the experiment is built and controlled.

ORIGINAL: ISAAC GYM

HORA task + custom simulation loop



Isaac Gym API → physics + robot-state arrays



CURRENT: ISAAC LAB

HORA task inside a reusable environment framework



Isaac Sim → scene assets + physics + sensors

Who runs each step?

Repository code advances physics and refreshes state.

The framework runs the loop; task functions supply behavior.

How is the world described?

Create simulation actors and operate on state arrays.

Configure robot, object and sensor components in a scene.

What does it enable here?

A working platform for learning in-hand rotation.

The same learning design, plus integrated sensing and data capture.

The learning method is retained; the surrounding simulation architecture is rebuilt.

The migration changes data flow, timing and configuration

The same robot-learning experiment must be expressed through a different software interface.

CONCEPT	ISAAC GYM: ORIGINAL	ISAAC LAB: CURRENT PORT
Control timing	Custom loop repeats physics updates for each action.	Framework repeats the action hook: 6 physics steps per decision.
Robot measurements	Explicitly refresh shared state arrays before reading them.	Read robot/object data through components; some values update on demand.
Physics settings	Settings collected in the original simulation configuration.	Translate settings across the scene, individual robots and objects.
Learning interface	Learner expects observations, reward, done and extra data.	A compatibility adapter restores that format and preserves timeout information.

Preserved interface: 16 finger commands, the same observation format, and unchanged learning code.

Why the port was difficult: hidden assumptions changed

Documented migration issues show why identical-looking code can produce different motion, contacts or images.

CHANGED ASSUMPTION	DEVELOPMENT DIFFICULTY	IMPLEMENTED FIX
Joint order	The imported hand lists joints in a different order; commands need remapping.	Reorder measurements and actions so each value belongs to the correct joint.
State-update timing	Two joint-position reads can return the same state, giving a zero velocity estimate.	Save the previous physics-step position explicitly before estimating velocity.
Contact reporting	A combined fingertip sensor returned zero filtered forces in parallel environments.	Use one object-filtered contact sensor per fingertip; this also supports GPU grasp checks.
Asset conversion	The robot-description import path could produce invisible object geometry.	Create simple shapes directly; use a separate mesh-to-scene conversion path.

Migration preserves the task’s intent; numerical equivalence and performance still need validation.

Evidence: comments and fixes in allegro_hand_hora_env.py, allegro_hand_grasp_env.py, deploy_ros2*.py and dataset collectors.